

Guía del alumno

Módulo 4, Minería de datos

Análisis Avanzado de Datos con Python

Cómo usar esta guía

Esta guía es para leer después de la clase. Trae el mismo código y las mismas cifras que viste en el laboratorio, con la explicación escrita y la solución de cada ejercicio.

Este módulo se dicta en dos clases. Los bloques 1 a 3 son de la primera y los bloques 4 a 7 de la segunda. Cuando vuelvas a abrir el notebook, ejecútalo desde arriba, porque cada bloque usa lo que armó el anterior.

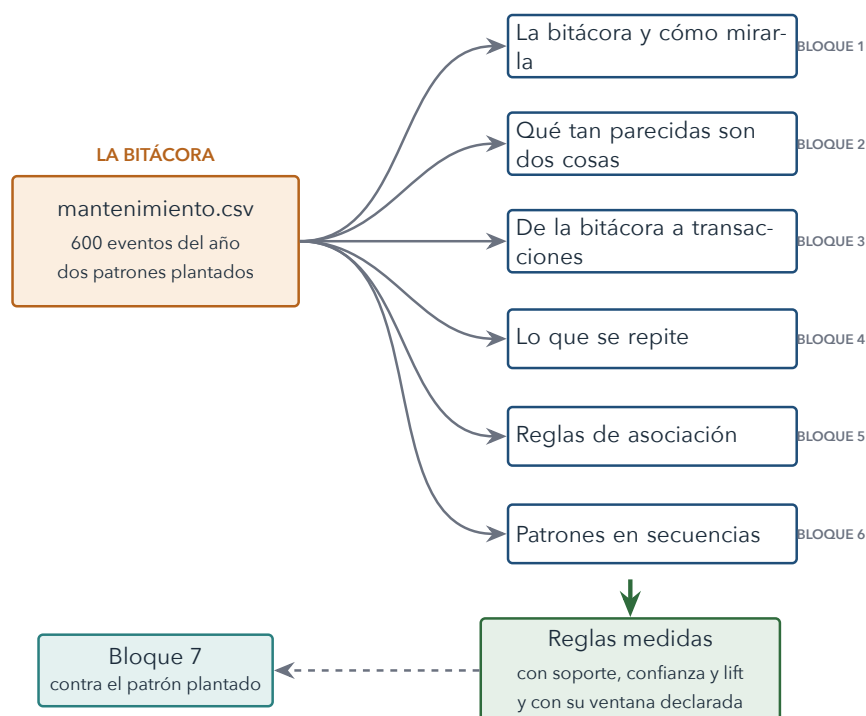
Lo que vas a poder hacer al terminar

- Mirar una bitácora de eventos de cuatro formas distintas, y saber qué habilita cada una.
- Elegir una medida de distancia sabiendo si te importa el tamaño o la forma.
- Convertir eventos sueltos en transacciones, que es lo que reciben estos algoritmos.
- Sacar combinaciones frecuentes con apriori y reglas con `association_rules`.
- Leer una regla con sus tres números y saber cuál de ellos engaña.
- Medir un patrón con su ventana declarada, y entender por qué sin ventana no sirve.

Todas las cifras de esta guía salen de la corrida verificada del notebook. La bitácora se genera dentro del notebook con semilla fija, así que tus cifras son las mismas. Los datos son sintéticos, y donde una cifra del laboratorio se aleja de la realidad del sistema chileno, la guía lo dice.

1. El módulo en una imagen

Llega una bitácora de 600 eventos de mantenimiento. Adentro hay dos patrones que nadie escribió. Al final del módulo están medidos, y además está escrito con qué ventana.



La imagen del libro de novedades

En toda instalación que funciona veinticuatro horas hay un cuaderno donde el turno que sale anota lo que pasó. Una línea por novedad, con la hora y qué se hizo. Nadie lo escribe pensando en analizarlo.

Al cabo de un año ese cuaderno tiene seiscientas líneas, y adentro hay un conocimiento que nadie escribió en ninguna parte. El maestro que lleva veinte años lo sabe. Sabe que cuando entra viento fuerte hay que revisar los mismos tres equipos. Sabe que después de cierta falla viene trabajo el fin de semana. Ese señor se va a jubilar.

Minería de datos es sacar eso del cuaderno y ponerlo en números, para que no dependa de que el maestro esté. No es reemplazarlo. Es escribir lo que él sabe, en una forma que se pueda revisar, discutir y corregir.

Las tres decisiones del módulo

Este es el primer módulo de aprendizaje automático del curso y conviene decirlo de entrada. No se implementa nada a mano, todo son llamadas a una biblioteca. Lo difícil no es el algoritmo.

Hay tres decisiones que toma el analista y no el algoritmo, y cambiar cualquiera cambia el resultado.

1. **Qué quiere decir que dos cosas se parezcan.** Bloque 2.
2. **Qué es una transacción.** Bloque 3.
3. **Cuál es la ventana en la que se mide el patrón.** Bloque 7.

2. Bloque 1, la bitácora y cómo mirarla

Qué se vio

Hasta acá el curso trabajó con tablas de números. La bitácora no es eso, cada fila es un hecho con una etiqueta de texto y una fecha, y no hay nada que promediar.

Los mismos 600 eventos se pueden mirar de cuatro formas, y cada una habilita preguntas distintas.

LOS MISMOS 600 EVENTOS, CUATRO FORMAS DE MIRARLOS

CONJUNTOS

Qué tipos tuvo cada central en cada mes.

{correctivo, inspeccion}

Sirve para reglas de asociación.

VECTORES

Cada central, 24 números, su día promedio.

[0, 0, 0, ..., 307.4, ...]

Sirve para medir distancias.

MATRIZ

Centrales en las filas, tipos en las columnas.

crosstab(central, tipo)

Sirve para ver concentraciones.

SECUENCIAS

Ordenados por fecha, qué vino después.

groupby(central).shift(-1)

Sirve cuando el orden importa.

La tabla es una sola. Lo que cambia es la pregunta que se puede hacer. Un conjunto no sabe de orden, una secuencia sí. Un vector no sabe de categorías, una matriz de conteos no sabe de magnitudes. Elegir la forma es elegir qué patrones vas a poder encontrar.

El código, paso a paso

1.1 La bitácora

```
import pandas as pd

# La bitácora del año, un evento por fila. Esto es lo que llena el turno a mano.
mant = pd.read_csv("mantenimiento.csv", parse_dates=["fecha"])
centrales = pd.read_csv("centrales.csv")

print(mant.shape)
mant.head()
```

```
(600, 4)
      central  fecha  tipo_evento  duracion_horas
0  Eolica Campo Abierto  2024-01-01  preventivo           5
1  Parque Solar Quebrada Honda  2024-01-01  preventivo           8
2  Parque Solar Llano Seco  2024-01-02  falla_mecanica          49
3  Parque Solar Quebrada Honda  2024-01-02  inspeccion           6
4    Central Tres Saltos  2024-01-03  correctivo          38
```

Seiscientos eventos del año, uno por fila. La columna que importa, tipo_evento, es texto. Eso es lo nuevo del módulo.

1.2 Qué tipos hay

```
# El diagnóstico del Módulo 3, ahora sobre una tabla de eventos.
print(mant["tipo_evento"].value_counts())
```

```
tipo_evento
preventivo      158
correctivo      118
inspeccion       99
falla_mecanica   98
evento_climatico  80
falla_electrica  47
Name: count, dtype: int64
```

Ciento cincuenta y ocho preventivos y, en el otro extremo, cuarenta y siete fallas eléctricas. Guarda ese cuarenta y siete, vuelve en el bloque 7.

1.3 Cuánto dura cada cosa

```
# Cuánto dura cada tipo. Una inspección son horas, una falla mecánica son días.
print(mant.groupby("tipo_evento")["duracion_horas"].agg(["count", "mean", "max"]).round(1))
```

```
            count  mean  max
tipo_evento
correctivo      118  38.9   72
evento_climatico  80  21.2   36
falla_electrica   47  22.4   48
falla_mecanica    98  50.6   94
inspeccion       99   4.7    8
preventivo      158  14.5   24
```

Una inspección son cuatro o cinco horas y una falla mecánica cincuenta de promedio, con un máximo de noventa y cuatro. Las etiquetas no son intercambiables.

1.4 Forma 1, conjuntos

```
# Forma 1, conjuntos. Que tipos de evento tuvo cada central en cada mes.
mant["mes"] = mant["fecha"].dt.month
conjuntos = mant.groupby(["central", "mes"])["tipo_evento"].agg(lambda s: sorted(set(s)))

print(len(conjuntos), "conjuntos, uno por central y mes")
print(conjuntos.head(4).to_string())
```

```
215 conjuntos, uno por central y mes
central      mes
Carboelectrica Costa Brava  1      [preventivo]
                             2  [falla_electrica, preventivo]
                             3  [correctivo, inspeccion]
                             4      [preventivo]
```

Doscientos quince conjuntos, uno por central y mes. El set bota los repetidos, así que si una central tuvo tres preventivos en un mes, el conjunto dice preventivo una sola vez. Esa pérdida es deliberada, porque las reglas de asociación trabajan con presencia, no con cantidad.

1.5 Forma 2, vectores

```
# Forma 2, vectores. Cada central es un vector de 24 números, su día promedio.
gen = pd.read_csv("generacion.csv", parse_dates=["fecha"])
perfil = gen.pivot_table(index="central", columns="hora", values="mwh", aggfunc="mean")

print(perfil.shape, "veinte centrales, cada una con 24 horas")
print(perfil.iloc[:3, 10:15].round(1))
```

```
(20, 24) veinte centrales, cada una con 24 horas
hora          10      11      12      13      14
central
Carboelectrica Costa Brava  306.8  307.2  307.4  306.8  307.3
Carboelectrica Roca Gris    223.9  224.3  224.0  224.2  224.1
Central Aguas Claras        98.9   99.0   98.7   98.8   98.9
```

Cada central es una fila de 24 números, su día promedio. Un vector se puede restar de otro, que es lo que hace falta para medir distancias.

1.6 Forma 3, matriz

```
# Forma 3, matriz. Centrales en las filas, tipos de evento en las columnas.
conteos = pd.crosstab(mant["central"], mant["tipo_evento"])

print(conteos.iloc[:4, :4].to_string())
```

```
tipo_evento          correctivo  evento_climatico  falla_electrica  falla_mecanica
central
Carboelectrica Costa Brava         10              1              2              2
Carboelectrica Roca Gris           7              1              2              4
Central Aguas Claras                4              2              3              5
Central Rio Manso Alto             10              1              2              1
```

Centrales en las filas, tipos en las columnas, y adentro la cuenta. Compárala con el conjunto, la matriz sabe que hubo diez correctivos y el conjunto solo sabía que hubo correctivo.

1.7 Forma 4, secuencias

```
# Forma 4, secuencias. El orden importa, así que se ordena y se mira el siguiente.
ordenados = mant.sort_values(["central", "fecha"]).copy()
ordenados["siguiente"] = ordenados.groupby("central")["tipo_evento"].shift(-1)

print(ordenados[["central", "fecha", "tipo_evento", "siguiente"]].head(5).to_string(index=False))
```

```
central  fecha  tipo_evento  siguiente
Carboelectrica Costa Brava 2024-01-21  preventivo  falla_electrica
Carboelectrica Costa Brava 2024-02-10  falla_electrica  preventivo
Carboelectrica Costa Brava 2024-02-18  preventivo  inspeccion
Carboelectrica Costa Brava 2024-03-12  inspeccion  correctivo
Carboelectrica Costa Brava 2024-03-24  correctivo  preventivo
```

Se ordena por central y fecha, y `shift(-1)` trae el evento siguiente a la misma fila. El `groupby` antes del `shift` es obligatorio. Sin él, el último evento de una central traería el primero de la siguiente.

La solución del ejercicio

El ejercicio pedía cruzar la bitácora con la ficha de cada central.

```
mt = mant.merge(centrales, on="central")

print(pd.crosstab(mt["tecnologia"], mt["tipo_evento"]))
```

Las eólicas se llevan 55 de los 80 eventos climáticos, siendo 4 centrales de 20. También tienen 44 de las 98 fallas mecánicas. Los dos números están relacionados y el bloque 7 explica por qué.

Referencia rápida

Instrucción	Para qué sirve
<code>df["col"].value_counts()</code>	Cuántas filas tiene cada categoría
<code>df.groupby(g) ["c"].agg(...)</code>	Varios cálculos de una vez
<code>.agg(lambda s: sorted(set(s)))</code>	Convertir un grupo en un conjunto ordenado
<code>pd.crosstab(a, b)</code>	Tabla de conteos de dos categorías
<code>df.pivot_table(index=, columns=)</code>	Una fila por entidad, una columna por dimensión
<code>df.groupby(g) ["c"].shift(-1)</code>	El valor siguiente dentro de cada grupo

Errores comunes

Lo que aparece en pantalla	Qué pasó
El evento siguiente es de otra central	Falta el <code>groupby</code> antes del <code>shift</code> .
<code>KeyError</code> al pedir un conjunto	Los conjuntos tienen índice de dos niveles, central y mes, así que se piden con una tupla.
El conjunto trae repetidos	Falta el <code>set</code> . Con <code>sorted(s)</code> solos, los repetidos quedan.

Tres preguntas de repaso

1. Por qué la bitácora es distinta de las tablas de los módulos anteriores

Porque la columna que importa es una etiqueta de texto y no hay nada que promediar. El valor está en qué cosas aparecen juntas, no en cuánto suman.

2. Qué se pierde al pasar a conjuntos

La cantidad. El conjunto solo sabe si el tipo estuvo o no estuvo, no cuántas veces. Es deliberado, porque es lo que esperan las reglas de asociación.

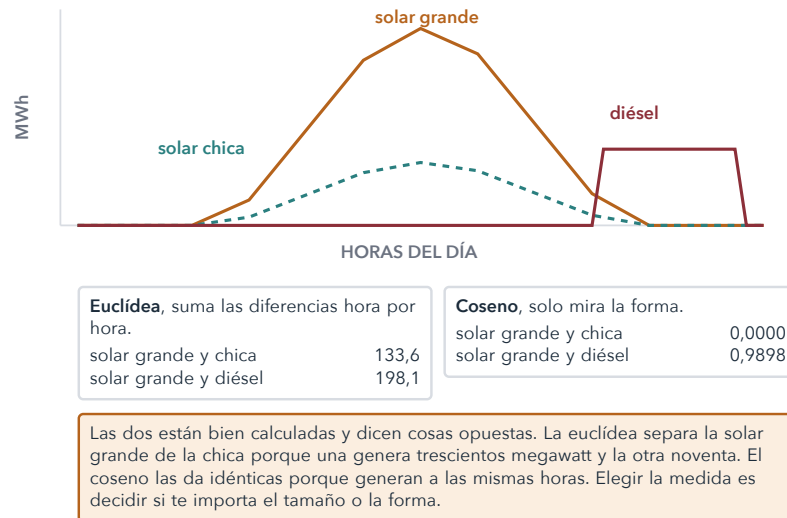
3. Por qué el `shift` va agrupado por central

Porque sin agrupar, el último evento de una central traería el primer evento de la siguiente, que es una transición que nunca ocurrió.

3. Bloque 2, qué tan parecidas son dos cosas

Qué se vio

Todo lo que viene después, agrupar, encontrar vecinos, recomendar, descansa sobre la idea de que dos cosas se parecen. Y parecerse no tiene una definición única.



El código, paso a paso

2.1 La euclídea

```
from scipy.spatial.distance import euclidean, cosine, jaccard

# Tres parques solares, uno grande, uno mediano y uno chico.
grande = perfil.loc["Parque Solar Altiplano"]
chico = perfil.loc["Parque Solar Quebrada Honda"]

# La euclídea suma las diferencias hora por hora. Mide tamaño.
print("euclídea entre la solar grande y la chica", round(euclidean(grande, chico), 1))
```

```
euclídea entre la solar grande y la chica 133.6
```

Ciento treinta y tres coma seis. Ese número solo no dice nada, las distancias se leen comparando. La euclídea suma las diferencias hora por hora, así que casi toda la distancia viene de que una central genera 300 MW y la otra 90.

2.2 El coseno

```
# El coseno mira solo la forma, no el tamaño. Cero quiere decir misma forma.
print(f"coseno entre la solar grande y la chica    {cosine(grande, chico):.4f}")
```

```
coseno entre la solar grande y la chica    0.0000
```

La misma pareja, otra medida, cero exacto. El coseno mira el ángulo entre los dos vectores, no su largo, y estos dos apuntan en la misma dirección.

2.3 El caso que descoloca

```
# Y acá viene lo que descoloca. El diésel solo genera de noche.
diesel = perfil.loc["Diesel Respaldo Cordillera"]

print(f"solar grande contra solar chica    euclídea {euclidean(grande, chico):7.1f}"
      f"    coseno {cosine(grande, chico):.4f}")
print(f"solar grande contra el diésel      euclídea {euclidean(grande, diesel):7.1f}"
      f"    coseno {cosine(grande, diesel):.4f}")
```

```
solar grande contra solar chica    euclídea    133.6    coseno 0.0000
solar grande contra el diésel      euclídea    198.1    coseno 0.9898
```

Acá está el bloque entero. Según la euclídea el diésel está a 198 y la solar chica a 133, o sea comparables. Según el coseno la solar chica está a 0 y el diésel a 0,9898, que es casi el máximo. El diésel genera de noche y la solar de día, son lo más opuesto de la tabla, y la euclídea no lo ve porque el diésel es chico.

2.4 Todas contra todas

```
from scipy.spatial.distance import pdist, squareform
import numpy as np

# Todas contra todas, de una vez. pdist devuelve la mitad, squareform la cuadra.
distancias = pd.DataFrame(squareform(pdist(perfil.values)),
                          index=perfil.index, columns=perfil.index)
# La diagonal es la distancia de cada central consigo misma y estorba.
distancias = distancias.mask(np.eye(len(distancias), dtype=bool))

print(distancias.iloc[:3, :3].round(1).to_string())
```

central	Carboelectrica Costa Brava	Carboelectrica Roca Gris	Central Aguas Claras
central			
Carboelectrica Costa Brava	NaN	407.7	1021.1
Carboelectrica Roca Gris	407.7	NaN	613.4
Central Aguas Claras	1021.1	613.4	NaN

pdist calcula todos los pares y squareform lo cuadra. La diagonal se pone en nulo, porque si no, la más parecida a cada central es ella misma.

2.5 La vecina más parecida

```
# Para cada central, cuál es la más parecida. idxmin da el nombre de la columna.
tec = centrales.set_index("central")["tecnologia"]
vecina = distancias.idxmin(axis=1).to_frame("mas_parecida")
vecina["misma_tecnologia"] = tec[vecina.index].values == tec[vecina["mas_parecida"]].values

print(vecina.head(5).to_string())
print("\nacierta la tecnología en", vecina["misma_tecnologia"].sum(), "de", len(vecina))
```

central	mas_parecida	misma_tecnologia
Carboelectrica Costa Brava	Termoelectrica Bahia Norte	False
Carboelectrica Roca Gris	Central Rio Manso Alto	False
Central Aguas Claras	Eolica Campo Abierto	False
Central Rio Manso Alto	Carboelectrica Roca Gris	False
Central Salto Verde	Termoelectrica Puerto Sur	False

acierta la tecnología en 7 de 20

Siete de veinte. Con seis tecnologías, el azar daría cerca de cuatro, así que la euclídea sobre megawatt crudos es apenas mejor que tirar una moneda.

2.6 Normalizar

```
# Siete de veinte es apenas mejor que el azar. El problema es la escala.
# Si cada perfil se divide por su máximo, todos quedan entre cero y uno.
normal = perfil.div(perfil.max(axis=1), axis=0)

d2 = pd.DataFrame(squareform(pdist(normal.values)), index=normal.index, columns=normal.index)
d2 = d2.mask(np.eye(len(d2), dtype=bool))
vecina2 = d2.idxmin(axis=1)
aciertos = (tec[vecina2.index].values == tec[vecina2.values].values).sum()

print("con el perfil normalizado acierta en", aciertos, "de", len(vecina2))
```

con el perfil normalizado acierta en 15 de 20

Quince de veinte. Una línea, dividir cada perfil por su propio máximo, y el acierto se duplica. Esta es la moraleja del bloque. Cuando las escalas son distintas, normalizar antes de medir no es un refinamiento, es la diferencia entre que funcione y que no.

2.7 Jaccard, para conjuntos

```
# Jaccard es para conjuntos. Mide qué proporción de los elementos no comparten.
uno = set(conjuntos[("Eolica Campo Abierto", 11)])
otro = set(conjuntos[("Termoelectrica Valle Central", 12)])

print("eólica en noviembre ", ", ", ".join(sorted(uno)))
print("térmica en diciembre ", ", ", ".join(sorted(otro)))
print("comparten", len(uno & otro), "de", len(uno | otro), "tipos, jaccard",
      round(1 - len(uno & otro) / len(uno | otro), 3))
```

```
eólica en noviembre   correctivo, evento_climatico, falla_electrica, falla_mecanica, inspeccion, preventivo
térmica en diciembre  preventivo
comparten 1 de 6 tipos, jaccard 0.833
```

Jaccard no necesita números. Noviembre de la eólica tuvo los seis tipos y diciembre de la térmica tuvo uno. Comparten uno de seis, así que la distancia es uno menos un sexto, 0,833.

La solución del ejercicio

El ejercicio pedía repetir la matriz con distancia coseno.

```
d3 = pd.DataFrame(squareform(pdist(perfil.values, metric="cosine")),
                  index=perfil.index, columns=perfil.index)
np.fill_diagonal(d3.values, np.nan)
v3 = d3.idxmin(axis=1)

print("aciertos", (tec[v3.index].values == tec[v3.values].values).sum(), "de", len(v3))
```

Sube bastante respecto de los siete de la euclídea cruda, porque el coseno ignora el tamaño, que era justo lo que estorbaba. No da exactamente lo mismo que normalizar por el máximo, porque normalizar por el máximo y normalizar por el largo del vector no son la misma operación.

OJO CON ESTAS DISTANCIAS

Entre dos parques solares del curso el coseno da 0,0000 exacto, y entre la hidro y la térmica también. Eso es un artefacto de cómo se generaron estos datos, con un perfil por tecnología escalado por potencia, así que dos solares tienen literalmente la misma forma.

En el Sistema Eléctrico Nacional dos parques solares tienen curvas parecidas pero no idénticas, porque dependen de la nubosidad, de la orientación de los paneles y de las restricciones de la línea que los evacúa. El coseno daría un número chico, no cero.

Referencia rápida

Instrucción	Para qué sirve
<code>euclidean(a, b)</code>	Distancia que mide tamaño
<code>cosine(a, b)</code>	Distancia que mide forma, 0 es idéntico
<code>jaccard(a, b)</code>	Distancia entre conjuntos, sobre vectores binarios
<code>pdist(M)</code>	Todos los pares, en formato comprimido
<code>pdist(M, metric="cosine")</code>	Lo mismo con otra métrica
<code>squareform(d)</code>	Pasa el formato comprimido a matriz cuadrada
<code>np.fill_diagonal(m, np.nan)</code>	Anula la diagonal, obligatorio antes de buscar vecinos
<code>df.idxmin(axis=1)</code>	El nombre de la columna con el mínimo de cada fila
<code>df.div(df.max(axis=1), axis=0)</code>	Normalizar cada fila por su propio máximo

Errores comunes

Lo que aparece en pantalla	Qué pasó
Cada central es su propia vecina	Falta anular la diagonal con <code>np.fill_diagonal</code> .
La distancia coseno da 1 entre cosas iguales	Estás pensando en la similitud. <code>scipy</code> entrega la distancia, donde 0 es idéntico.
Todo queda agrupado por tamaño	Estás usando euclídea sobre datos sin normalizar, con escalas muy distintas.
<code>ValueError</code> en <code>jaccard</code>	Jaccard espera vectores booleanos o de ceros y unos, no números cualesquiera.

Tres preguntas de repaso

1. Qué mide cada una de las tres distancias

La euclídea mide tamaño, porque suma las diferencias punto a punto. El coseno mide forma, porque compara direcciones. Jaccard mide cuánto comparten dos conjuntos.

2. Por qué la distancia coseno es cero cuando dos cosas son iguales

Porque es una distancia, no una similitud. Cero es idéntico y dos es lo más opuesto. La similitud coseno es uno menos esto.

3. Qué cambia normalizar antes de medir

Saca del medio la escala. Acá el acierto de la vecina más parecida sube de 7 de 20 a 15 de 20 con una sola línea.

4. Bloque 3, de la bitácora a transacciones

Qué se vio

Los algoritmos que buscan patrones en etiquetas no reciben una bitácora, reciben una lista de transacciones. Y en la bitácora no hay transacciones, hay eventos sueltos con fecha. Somos nosotros los que decidimos qué eventos cuentan como que pasaron juntos.

LA BITÁCORA, UNA FILA POR EVENTO

central fecha tipo_evento
Costa Brava 2024-02-10 falla_electrica
Costa Brava 2024-02-18 preventivo

groupby
central, mes

una transacción
[falla_electrica, preventivo]

Y LA TRANSACCIÓN, COMO TABLA BINARIA

correctivo	climatico	electrica	mecanica	inspeccion	preventivo
False	False	True	False	False	True

Decidir qué es una transacción es la decisión del módulo. Acá elegimos central y mes, y salen 215 transacciones de 2,17 tipos en promedio. Por central y trimestre serían menos y más grandes, y todos los soportes subirían. El algoritmo no elige esto, lo eliges tú.

El código, paso a paso

3.1 La decisión

```
# Antes de buscar patrones hay que decidir qué es una transacción.
# Acá elegimos central y mes. Todo lo que le pasó a una central en un mes.
transacciones = mant.groupby(["central", "mes"])["tipo_evento"].agg(lambda s: sorted(set(s)))

print(len(transacciones), "transacciones")
print("tipos distintos por transacción, en promedio", round(transacciones.apply(len).mean(), 2))
```

```
215 transacciones
tipos distintos por transacción, en promedio 2.17
```

Doscientas quince transacciones, con 2,17 tipos distintos en promedio. Ese promedio bajo importa, porque con transacciones tan chicas las reglas de tres o cuatro elementos van a ser raras.

3.2 Cómo se ve una transacción

```
# Así se ve una transacción. Es una lista de etiquetas, no números.
print(transacciones.head(6).to_string())
```

```
central      mes
Carboelectrica Costa Brava 1      [preventivo]
                             2      [falla_electrica, preventivo]
                             3      [correctivo, inspeccion]
                             4      [preventivo]
                             5      [correctivo, evento_climatico]
                             6      [correctivo, falla_mecanica]
```

Costa Brava en enero tuvo un preventivo, en febrero un preventivo y una falla eléctrica. Una transacción es una lista de etiquetas, sin números.

3.3 La tabla binaria

```
from mlxtend.preprocessing import TransactionEncoder

# El encoder convierte las listas en una tabla de verdaderos y falsos.
# Una columna por tipo de evento, una fila por transacción.
codificador = TransactionEncoder()
binaria = pd.DataFrame(codificador.fit_transform(transacciones.tolist()),
                       columns=codificador.columns_)

print(binaria.shape)
print(binaria.head(4).to_string())
```

```
(215, 6)
   correctivo  evento_climatico  falla_electrica  falla_mecanica  inspeccion  preventivo
0         False              False              False              False         False         True
1         False              False              True              False         False         True
2          True              False              False              False         True         False
3         False              False              False              False         False         True
```

El codificador convierte cada lista en una fila de verdaderos y falsos. Es la misma idea de `get_dummies`, entregada en el formato que esperan los algoritmos de `mlxtend`.

3.4 Los soportes

```
# El soporte de un tipo es la proporción de transacciones donde aparece.
print(binaria.mean().sort_values(ascending=False).round(3))
```

```
preventivo      0.563
correctivo      0.400
inspeccion      0.395
falla_mecanica  0.363
evento_climatico 0.256
falla_electrica 0.195
dtype: float64
```

El preventivo aparece en el 56,3 por ciento de las transacciones y la falla eléctrica en el 19,5. Guarda esos dos números, porque el bloque 5 explica con ellos por qué existe el lift.

La solución del ejercicio

El ejercicio pedía cambiar el mes por el trimestre.

```
mant["trimestre"] = mant["fecha"].dt.quarter
por_trimestre = mant.groupby(["central", "trimestre"])["tipo_evento"].agg(
    lambda s: sorted(set(s)))

print("por trimestre", len(por_trimestre), "transacciones, tamaño medio",
      round(por_trimestre.apply(len).mean(), 2))
```

Quedan menos transacciones y más grandes, porque cada una junta tres meses. Y con eso suben todos los soportes. Es el mismo archivo dando otro resultado por una decisión nuestra.

Referencia rápida

Instrucción	Para qué sirve
<code>dt.month · dt.quarter</code>	Mes y trimestre, para definir la ventana
<code>dt.isocalendar().week</code>	Semana del año
<code>TransactionEncoder()</code>	El codificador de <code>mlxtend</code>
<code>te.fit_transform(lista)</code>	De lista de listas a matriz de booleanos
<code>te.columns_</code>	Los nombres de las columnas que armó
<code>binaria.mean()</code>	El soporte de cada elemento por separado

Tres preguntas de repaso

1. Cuál es la ventana correcta para una transacción

No hay una. Se elige según la pregunta y según lo que uno sabe del proceso, y lo importante es escribirla y declararla junto al resultado.

2. Qué pasaría con una transacción por central, sin tiempo

Cada central tendría los seis tipos, todas las transacciones serían iguales, el soporte de todo sería uno y no habría nada que encontrar.

3. Por qué importa el soporte individual del preventivo

Porque está en más de la mitad de las transacciones, así que cualquier regla que apunte a preventivo va a tener confianza alta sin que eso signifique nada.

5. Bloque 4, lo que se repite

Qué se vio

Con seis tipos de evento hay 63 combinaciones posibles. Con veinte serían más de un millón. Apriori las recorre podando, con una idea simple, si una combinación de dos no es frecuente, ninguna que la contenga puede serlo.

Lo que no resuelve el algoritmo es el umbral de soporte. Ese número lo pones tú.

El código, paso a paso

4.1 Apriori

```
import warnings
from mlxtend.frequent_patterns import apriori

# Al importarse, mlxtend enciende todos los avisos de deprecación de Python, y
# en Colab eso llena la pantalla con avisos del propio Jupyter. Se vuelve a
# dejar el comportamiento por defecto. Una biblioteca puede cambiar una
# configuración global, y conviene saber mirarlo.
warnings.filterwarnings("ignore", category=DeprecationWarning)

# apriori busca todas las combinaciones que aparecen en al menos el 5 por ciento
# de las transacciones. No hay que programar nada, es una llamada.
frecuentes = apriori(binaria, min_support=0.05, use_colnames=True)
frecuentes["n"] = frecuentes["itemsets"].apply(len)

print(len(frecuentes), "combinaciones frecuentes")
print(frecuentes["n"].value_counts().sort_index().to_string())
```

```
29 combinaciones frecuentes
n
1      6
2     14
3      9
```

Una línea. Busca todo lo que aparece en al menos el 5 por ciento de las 215 transacciones, o sea en once o más. Salen 29 combinaciones, de las cuales 6 son de un solo elemento.

POR QUÉ ESA LÍNEA DE WARNINGS

Al importarse, `mlxtend` enciende todos los avisos de deprecación de Python, que normalmente están apagados. En Colab eso hace que aparezca en pantalla una parrafada de avisos del propio Jupyter, que no tienen nada que ver con el lab.

La línea devuelve el comportamiento por defecto. Vale la pena saberlo, porque una biblioteca puede cambiar una configuración global del proceso y después cuesta entender de dónde salió lo que ves.

4.2 Las parejas

```
# Las parejas más frecuentes. Estas son las que hay que mirar.
pares = frecuentes[frecuentes["n"] == 2].sort_values("support", ascending=False)

print(pares.head(6).to_string(index=False))
```

```
support            itemsets  n
0.200000      (preventivo, inspeccion) 2
0.186047      (correctivo, preventivo) 2
0.176744 (falla_mecanica, evento_climatico) 2
0.172093      (preventivo, falla_mecanica) 2
0.162791      (correctivo, falla_electrica) 2
0.134884      (correctivo, inspeccion) 2
```

Las dos primeras son las aburridas, porque juntan cosas frecuentes con cosas frecuentes. Fíjate en la tercera, evento climático con falla mecánica, y en la quinta, correctivo con falla eléctrica.

4.3 FP-Growth

```
from mlxtend.frequent_patterns import fpgrowth

# FP-Growth es otro algoritmo, más rápido en tablas grandes. Da lo mismo.
frecuentes_fp = fpgrowth(binaria, min_support=0.05, use_colnames=True)

print("apriori encuentra ", len(frecuentes))
print("fpgrowth encuentra", len(frecuentes_fp))
```

```
apriori encuentra 29
fpgrowth encuentra 29
```

Otro algoritmo, otra estrategia interna, mucho más rápido en tablas grandes. Devuelve las mismas 29 combinaciones. Se elige por velocidad, no por calidad del resultado.

4.4 El umbral

```
# El umbral es la decisión del bloque. Bajarlo encuentra más y casi todo ruido.
for umbral in (0.03, 0.05, 0.10, 0.20):
    print(umbral, "->", len(apriori(binaria, min_support=umbral, use_colnames=True)),
          "combinaciones")
```

```
0.03 -> 40 combinaciones
0.05 -> 29 combinaciones
0.1 -> 17 combinaciones
0.2 -> 6 combinaciones
```

De 40 combinaciones con el 3 por ciento a 6 con el 20. La misma tabla y el mismo algoritmo, y el resultado cambia por completo según un número que pusiste tú. Con 215 transacciones, el 3 por ciento son seis casos, que ya es demasiado poco para confiar.

La solución del ejercicio

El ejercicio pedía ver los tríos.

```
trios = frecuentes[frecuentes["n"] == 3].sort_values("support", ascending=False)

print(trios.head(4).to_string(index=False))
```

Casi todos los tríos contienen preventivo, que es el tipo más frecuente. Ese es exactamente el problema que el lift viene a resolver en el bloque siguiente.

Referencia rápida

Instrucción	Para qué sirve
<code>apriori(b, min_support=, use_colnames=)</code>	Combinaciones frecuentes
<code>fpgrowth(b, min_support=)</code>	Lo mismo, más rápido en tablas grandes
<code>df["itemsets"].apply(len)</code>	El tamaño de cada combinación
<code>use_colnames=True</code>	Devuelve los nombres en vez de los números de columna

Errores comunes

Lo que aparece en pantalla	Qué pasó
Las combinaciones salen como números	Falta <code>use_colnames=True</code> .
<code>DeprecationWarning</code> sobre el tipo de la tabla	<code>mlxtend</code> espera booleanos. Si la tabla trae ceros y unos, conviértela con <code>astype(bool)</code> .
El resultado está vacío	El umbral quedó demasiado alto para el tamaño de las transacciones.

Tres preguntas de repaso

1. Qué significa el soporte

La proporción de transacciones donde aparece la combinación completa. Es una frecuencia, nada más.

2.Cuál es la idea que usa apriori para podar

Que un superconjunto nunca puede ser más frecuente que sus partes. Si una pareja no llega al umbral, ningún trío que la contenga va a llegar.

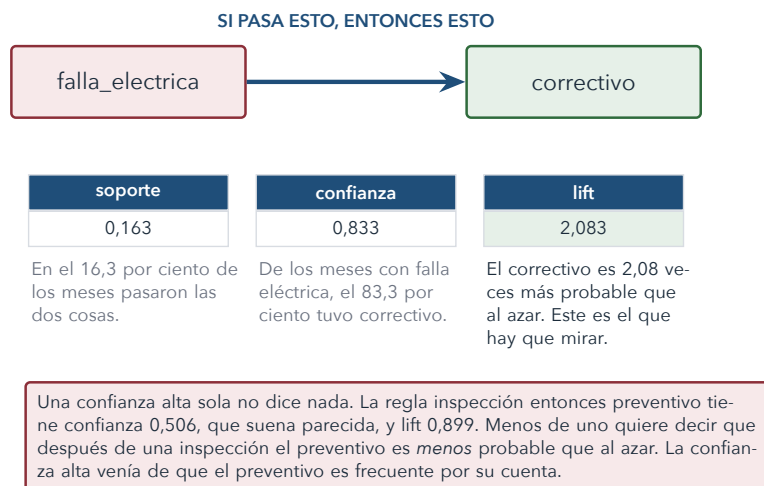
3. Por qué una combinación frecuente todavía no es una regla

Porque no tiene dirección. Sabe que dos cosas aparecen juntas, no sabe cuál lleva a cuál.

6. Bloque 5, reglas de asociación

Qué se vio

Una regla dice si pasa esto, entonces pasa esto otro, y trae tres números. La gente mira el equivocado.



El código, paso a paso

5.1 Las reglas

```
from mlxtend.frequent_patterns import association_rules

# Una combinación frecuente no es una regla. La regla tiene dirección,
# si pasa esto entonces pasa esto otro.
reglas = association_rules(frecuentes, metric="confidence", min_threshold=0.5)
reglas["si"] = reglas["antecedents"].apply(lambda s: ", ".join(sorted(s)))
reglas["entonces"] = reglas["consequents"].apply(lambda s: ", ".join(sorted(s)))
COLS = ["si", "entonces", "support", "confidence", "lift"]

print(len(reglas), "reglas con confianza de al menos 0,5")
```

```
16 reglas con confianza de al menos 0,5
```

Dieciséis reglas con confianza de al menos 0,5. Las columnas si y entonces son para poder leerlas, porque mlxtend las entrega como conjuntos congelados.

5.2 Las reglas simples

```
# Las reglas simples, de un evento a un evento, ordenadas por lift.
simples = reglas[(reglas["antecedents"].apply(len) == 1)
                  & (reglas["consequents"].apply(len) == 1)]

print(simples.sort_values("lift", ascending=False)[COLS].head(6).round(3).to_string(index=False))
```

	si	entonces	support	confidence	lift
	falla_electrica	correctivo	0.163	0.833	2.083
	evento_climatico	falla_mecanica	0.177	0.691	1.904
	falla_electrica	preventivo	0.102	0.524	0.931
	evento_climatico	preventivo	0.130	0.509	0.905
	inspeccion	preventivo	0.200	0.506	0.899

Ordenadas por lift. Las dos primeras están sobre 2 y las tres siguientes bajo 1. Esa línea, el uno, es la que separa una regla que sirve de una que no.

5.3 La trampa de la confianza

```
# La trampa de la confianza. Estas dos reglas tienen confianza parecida.
mirar = simples[simples["entonces"].isin(["correctivo", "preventivo"])]

print(mirar.sort_values("confidence",
↳ ascending=False)[COLS].head(6).round(3).to_string(index=False))
```

	si	entonces	support	confidence	lift
	falla_electrica	correctivo	0.163	0.833	2.083
	falla_electrica	preventivo	0.102	0.524	0.931
	evento_climatico	preventivo	0.130	0.509	0.905
	inspeccion	preventivo	0.200	0.506	0.899

Las confianzas van de 0,506 a 0,833, que no es tanta diferencia. Los lift van de 0,899 a 2,083, que sí. La razón es que el preventivo está en el 56 por ciento de las transacciones, así que apuntar a preventivo da confianza alta sin esfuerzo.

5.4 Las compuestas

```
# Reglas con más de un evento a la izquierda, que suelen tener el lift más alto.
compuestas = reglas[reglas["antecedents"].apply(len) > 1]

print(compuestas.sort_values("lift",
↳ ascending=False)[COLS].head(5).round(3).to_string(index=False))
```

	si	entonces	support	confidence	lift
	correctivo, falla_mecanica	falla_electrica	0.065	0.519	2.654
	falla_electrica, falla_mecanica	correctivo	0.065	1.000	2.500
	falla_mecanica, inspeccion	evento_climatico	0.074	0.593	2.316
	falla_mecanica, preventivo	evento_climatico	0.098	0.568	2.219
	evento_climatico, preventivo	falla_mecanica	0.098	0.750	2.067

Con dos eventos a la izquierda el lift sube. Falla eléctrica más falla mecánica lleva a correctivo con confianza 1, o sea siempre. Cuidado con ese uno, el soporte es 0,065, que sobre 215 transacciones son catorce casos.

La solución del ejercicio

El ejercicio pedía las reglas que terminan en correctivo.

```
hacia_correctivo = reglas[reglas["entonces"] == "correctivo"]

print(hacia_correctivo.sort_values("lift", ascending=False)[COLS].round(3).to_string(index=False))
```

Salen cuatro. La de falla eléctrica sola tiene lift 2,083 y buena cobertura. Las compuestas tienen lift más alto y muy poco soporte. Esa tensión, entre una regla fuerte y una que se aplica seguido, es real y no se resuelve con un número.

ESTO NO DICE QUE UNA COSA CAUSE LA OTRA

La regla dice que cuando en un mes hubo una falla eléctrica, en ese mismo mes hubo un correctivo el 83 por ciento de las veces. Eso es una coincidencia medida, no una causa.

Acá sabemos que sí hay causa, porque el patrón se plantó a propósito al generar el archivo. En un archivo de verdad no lo sabrías, y el mismo número podría salir porque las dos cosas dependen de un tercero, como que la central es vieja, o simplemente porque hay pocos casos.

La regla sirve para decidir dónde mirar, no para cerrar el caso.

Referencia rápida

Instrucción	Para qué sirve
<code>association_rules(f, metric=, min_threshold=)</code>	Saca las reglas de las combinaciones
<code>metric="lift"</code>	Filtrar por lift en vez de por confianza
<code>df["antecedents"].apply(len)</code>	Cuántos eventos hay a la izquierda
<code>", ".join(sorted(s))</code>	Convertir el conjunto congelado en texto legible
Métrica	Qué responde
Soporte	En qué proporción de todas las transacciones aparece la combinación completa
Confianza	De las transacciones con el antecedente, en cuántas apareció el consecuente
Lift	Cuántas veces más probable se vuelve el consecuente. Uno es que el antecedente no aporta nada

Tres preguntas de repaso

1. Por qué la confianza engaña

Porque puede venir enteramente de que el consecuente ya era frecuente por su cuenta. La regla inspección entonces preventivo tiene confianza 0,506 y lift 0,899.

2. Cómo se calcula el lift

Es la confianza dividida por el soporte del consecuente. Acá $0,833$ entre $0,400$ da $2,083$.

3. Qué hay que desconfiar de una confianza igual a 1

Del número de casos. Esa regla tiene soporte 0,065, o sea catorce transacciones. Catorce de catorce impresiona y no alcanza para concluir nada.

7. Bloque 6, patrones en secuencias

Qué se vio

Las reglas del bloque anterior no saben de orden. Dentro del mismo mes da lo mismo que el correctivo haya sido antes o después de la falla. Y lo que a todo el mundo le interesa es la sucesión.

El código, paso a paso

6.1 Los días hasta el siguiente

```
# Las reglas del bloque anterior no saben de orden. Dentro de un mes, da igual
# que el correctivo haya sido antes o después de la falla. Aquí sí importa.
ordenados["dias_hasta_siguiente"] = (ordenados.groupby("central")["fecha"].shift(-1)
                                     - ordenados["fecha"]).dt.days

print(ordenados[["central", "fecha", "tipo_evento", "siguiente",
                 "dias_hasta_siguiente"]].head(6).to_string(index=False))
```

	central	fecha	tipo_evento	siguiente	dias_hasta_siguiente
Carboelectrica Costa Brava	2024-01-21	preventivo	falla_electrica	20.0	
Carboelectrica Costa Brava	2024-02-10	falla_electrica	preventivo	8.0	
Carboelectrica Costa Brava	2024-02-18	preventivo	inspeccion	23.0	
Carboelectrica Costa Brava	2024-03-12	inspeccion	correctivo	12.0	
Carboelectrica Costa Brava	2024-03-24	correctivo	preventivo	32.0	
Carboelectrica Costa Brava	2024-04-25	preventivo	correctivo	11.0	

La misma tabla ordenada del bloque 1, ahora con cuántos días faltan hasta el evento siguiente de esa central. Los números son grandes porque cada central tiene unos treinta eventos al año.

6.2 La matriz de transición

```
# La matriz de transición. Cada fila suma uno, y dice qué viene después.
transicion = pd.crosstab(ordenados["tipo_evento"], ordenados["siguiente"],
                        normalize="index")

print(transicion.round(3).to_string())
```

siguiente	correctivo	evento_climatico	falla_electrica	falla_mecanica	inspeccion	preventivo
tipo_evento						
correctivo	0.219	0.061	0.079	0.140	0.167	0.333
evento_climatico	0.026	0.179	0.026	0.603	0.077	0.090
falla_electrica	0.787	0.000	0.064	0.000	0.043	0.106
falla_mecanica	0.168	0.221	0.084	0.063	0.189	0.274
inspeccion	0.149	0.170	0.074	0.106	0.170	0.330
preventivo	0.151	0.138	0.112	0.105	0.217	0.276

Cada fila suma uno y dice qué viene después. El `normalize="index"` es lo que hace eso. Sin él sería una tabla de conteos y las filas no se podrían comparar, porque hay tipos mucho más frecuentes que otros.

6.3 La fila que importa

```
# La fila que importa. Después de una falla eléctrica, casi siempre un correctivo.
print(transicion.loc["falla_electrica"].sort_values(ascending=False).round(3).to_string())
```

```
siguiente
correctivo      0.787
preventivo      0.106
falla_electrica 0.064
inspeccion      0.043
evento_climatico 0.000
falla_mecanica  0.000
```

Después de una falla eléctrica, el 78,7 por ciento de las veces el evento siguiente en esa central es un correctivo. Los dos ceros de abajo son interesantes, pero con 47 casos un cero observado no quiere decir imposible.

6.4 Cuánto se demora

```
# Y cuánto se demora. La mediana de días entre la falla y lo que viene después.
electricas = ordenados[ordenados["tipo_evento"] == "falla_electrica"]

print(electricas["dias_hasta_siguiente"].describe().round(1).to_string())
```

```
count    47.0
mean      5.0
std       8.7
min       0.0
25%       1.0
50%       2.0
75%       3.0
max      41.0
```

La mediana es 2 días y el 75 por ciento está dentro de 3. Eso es lo que convierte el patrón en algo accionable. El promedio es 5, arrastrado por una cola que llega a 41 días, así que acá la mediana es la que describe.

La solución del ejercicio

El ejercicio pedía comparar la fila de evento climático.

```
print(transicion.loc["evento_climatico"].sort_values(ascending=False).round(3))
```

Da 0,603 hacia falla mecánica, contra 0,787 de la eléctrica. La razón es que la falla mecánica plantada ocurre el mismo día que el evento climático, así que a veces queda anotada antes y el shift no la ve como siguiente. Es un límite real del método cuando hay eventos simultáneos.

Referencia rápida

Instrucción	Para qué sirve
<code>df.sort_values([g, "fecha"])</code>	Ordenar antes de mirar secuencias
<code>groupby(g) ["c"].shift(-1)</code>	El evento siguiente dentro del grupo
<code>pd.crosstab(a, b, normalize="index")</code>	Matriz de transición, cada fila suma uno
<code>.dt.days</code>	Convertir una diferencia de fechas en días

Tres preguntas de repaso

1. Qué le falta a una regla de asociación que sí tiene la matriz de transición

El orden. La regla solo sabe que las dos cosas estuvieron en la misma ventana.

2. Para qué sirve `normalize` igual a `index`

Para que cada fila sume uno, y así se puedan comparar filas de tipos con frecuencias muy distintas.

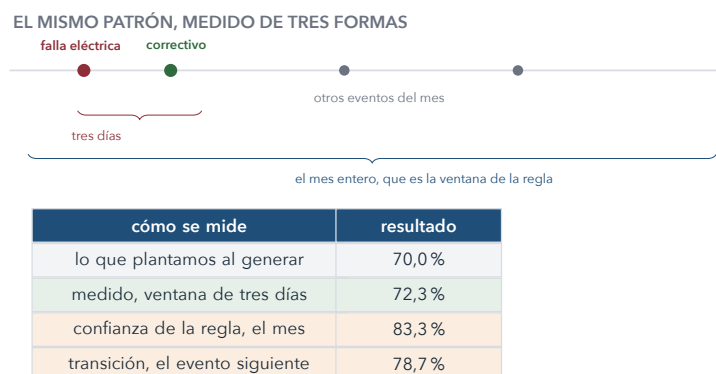
3. Por qué la fila del evento climático es más débil

Porque su patrón es del mismo día, y cuando dos eventos caen el mismo día el orden lo decide cómo quedaron en el archivo.

8. Bloque 7, contra la verdad

Qué se vio

Igual que en el Módulo 3, acá hay una ventaja que el trabajo real no da. Sabemos exactamente qué patrones se plantaron, porque se pusieron a propósito al generar el archivo.



Los tres números están bien y responden preguntas distintas. La regla mira el mes completo, así que cualquier correctivo del mes cuenta y el porcentaje sube. La transición mira solo el evento inmediatamente siguiente, sin límite de días. Cuando alguien muestre un porcentaje sacado de reglas de asociación, la primera pregunta es cuál era la ventana.

El código, paso a paso

7.1 El patrón 1, medido directo

```
# Patrón 1, medido directo. Para cada falla eléctrica, hubo un correctivo
# en la misma central dentro de los tres días siguientes.
electricas = mant[mant["tipo_evento"] == "falla_electrica"]
correctivos = mant[mant["tipo_evento"] == "correctivo"]

con_correctivo = 0
for fila in electricas.itertuples():
    cerca = correctivos[(correctivos["central"] == fila.central)
                        & (correctivos["fecha"] >= fila.fecha)
                        & (correctivos["fecha"] <= fila.fecha + pd.Timedelta(days=3))]
    con_correctivo += len(cerca) > 0

print(len(electricas), "fallas eléctricas")
print(con_correctivo, "con correctivo dentro de tres días =",
      round(100 * con_correctivo / len(electricas), 1), "por ciento")
```

```
47 fallas eléctricas
34 con correctivo dentro de tres días = 72.3 por ciento
```

Cuarenta y siete fallas eléctricas, treinta y cuatro con un correctivo dentro de tres días. 72,3 por ciento contra el 70 que se plantó. El bucle es feo a propósito, porque es la traducción literal de la pregunta.

7.2 El patrón 2

```
# Patrón 2, la mitad de los climáticos con una falla mecánica el mismo día.
climaticos = mant[mant["tipo_evento"] == "evento_climatico"]
mecanicas = mant[mant["tipo_evento"] == "falla_mecanica"]

con_mecanica = 0
for fila in climaticos.itertuples():
    mismo = mecanicas[(mecanicas["central"] == fila.central)
                      & (mecanicas["fecha"] == fila.fecha)]
    con_mecanica += len(mismo) > 0

print(len(climaticos), "eventos climáticos")
print(con_mecanica, "con falla mecánica el mismo día =",
      round(100 * con_mecanica / len(climaticos), 1), "por ciento")
```

```
80 eventos climáticos
43 con falla mecánica el mismo día = 53.8 por ciento
```

Ochenta eventos climáticos, cuarenta y tres con falla mecánica el mismo día. 53,8 contra 50. Los dos quedan un poco arriba porque, después de plantar los patrones, el archivo se rellena hasta 600 eventos y algunos cayeron cerca por casualidad. Eso es ruido de fondo, y en datos reales pasa igual.

7.3 Los cuatro números

```
# Los cuatro números del mismo patrón, uno al lado del otro.
regla = simples[(simples["si"] == "falla_electrica")
                & (simples["entonces"] == "correctivo")]

print(f"lo que plantamos al generar el archivo {70.0:5.1f} por ciento")
print(f"medido, con ventana de tres días      "
      f"{100 * con_correctivo / len(electricas):5.1f} por ciento")
print(f"la confianza de la regla, el mismo mes "
      f"{100 * float(regla['confidence'].iloc[0]):5.1f} por ciento")
print(f"la transición, el evento siguiente     "
      f"{100 * transicion.loc['falla_electrica', 'correctivo']:5.1f} por ciento")
```

```
lo que plantamos al generar el archivo    70.0 por ciento
medido, con ventana de tres días           72.3 por ciento
la confianza de la regla, el mismo mes     83.3 por ciento
la transición, el evento siguiente         78.7 por ciento
```

Acá está el remate del módulo. El patrón es uno solo y las tres mediciones dan distinto.

La solución del ejercicio

El ejercicio pedía mover la ventana.

```
for ventana in (1, 3, 7):
    cuenta = 0
    for fila in electricas.itertuples():
        cerca = correctivos[(correctivos["central"] == fila.central)
                             & (correctivos["fecha"] >= fila.fecha)
                             & (correctivos["fecha"] <= fila.fecha + pd.Timedelta(days=ventana))]
        cuenta += len(cerca) > 0
    print("ventana de", ventana, "dias ->", round(100 * cuenta / len(electricas), 1))
```

Con un día baja y con siete sube. Cuál pondrías en un informe. La respuesta correcta es cualquiera, con la ventana escrita al lado.

TRES NÚMEROS DISTINTOS, Y LOS TRES CORRECTOS

El patrón plantado es uno solo y las tres mediciones dan distinto, porque cada una responde una pregunta distinta.

La medición directa pregunta si hubo un correctivo dentro de tres días. La confianza de la regla pregunta si hubo un correctivo en el mismo mes, que es una ventana más ancha y por eso da más alto. La matriz de transición pregunta si el correctivo fue el evento inmediatamente siguiente, sin límite de días.

Cuando alguien te muestre un porcentaje sacado de reglas de asociación, la primera pregunta es cuál era la ventana. Sin eso el número no quiere decir nada.

Tres preguntas de repaso

1. Por qué la confianza de la regla da más que la medición a tres días

Porque la regla mira el mes completo, así que cualquier correctivo del mes cuenta, incluso uno que ocurrió veinte días después o antes.

2. Por qué las mediciones dan un poco más que lo plantado

Por el relleno. Después de plantar los patrones el archivo se completó con otros eventos, y algunos cayeron cerca por azar.

3. Qué hay que preguntar siempre frente a un porcentaje de este tipo

Cuál era la ventana. Sin ella el número no se puede comparar ni usar.

Hoja de referencia del Módulo 4

Para imprimir y tener al lado

MIRAR LA BITÁCORA

<code>value_counts()</code>	Cuántos hay de cada tipo
<code>groupby(g) ["c"].agg([...])</code>	Conteo, promedio y máximo de una vez
<code>.agg(lambda s: sorted(set(s)))</code>	El grupo como conjunto, sin repetidos
<code>pd.crosstab(a, b)</code>	Matriz de conteos
<code>groupby(g) ["c"].shift(-1)</code>	El evento siguiente, dentro del grupo

DISTANCIAS, LA PRIMERA DECISIÓN

<code>euclidean(a, b)</code>	Mide tamaño
<code>cosine(a, b)</code>	Mide forma. 0 es idéntico, no 1
<code>jaccard(a, b)</code>	Para conjuntos, sobre vectores binarios
<code>squareform(pdist(M, metric=))</code>	Todos contra todos
<code>np.fill_diagonal(m, np.nan)</code>	Siempre, antes de buscar vecinos
<code>M.div(M.max(axis=1), axis=0)</code>	Normalizar. Acá subió de 7 a 15 de 20

TRANSACCIONES, LA SEGUNDA DECISIÓN

<code>groupby([ent, ventana])</code>	Qué cuenta como que pasó junto
<code>TransactionEncoder().fit_transform</code>	De listas a tabla binaria
<code>binaria.mean()</code>	El soporte de cada elemento

PATRONES Y REGLAS

<code>apriori(b, min_support=, use_colnames=)</code>	Combinaciones frecuentes
<code>fpgrowth(b, min_support=)</code>	Igual resultado, más rápido
<code>association_rules(f, metric=, min_threshold=)</code>	Las reglas, con dirección
<code>pd.crosstab(a, b, normalize="index")</code>	Matriz de transición

Las cinco cosas que hay que recordar del módulo

- Los mismos datos admiten cuatro formas, y cada una habilita otras preguntas.
- Parecerse no tiene una sola definición, y con escalas distintas hay que normalizar.
- Qué cuenta como una transacción lo decides tú, y es lo que más cambia el resultado.
- De los tres números de una regla, el lift es el único que dice si el antecedente aporta.
- Un porcentaje sacado de reglas de asociación, sin su ventana, no se puede usar.